For

1CS201

OBJECT ORIENTED PROGRAMMING

B. Tech. Semester II

Report Prepared By

Krish Shah (23BCE152)

Malav Sheth (23BCE165)

School of Engineering

Institute of Technology

Nirma University

Ahmedabad

382481

Index

1. Introduction(Description of Topic)
2. Code
3. Output
4. Conclusion

INTRODUCTION

This code implements a Personal Health Record System for managing users and their health records, allowing users to track medications and doctor consultations. The system comprises several key classes and services, organized to handle user information, validate inputs, and manage file-based record storage.

CODE

```java
import java.io.*;
import java.util.*;
import java.util.regex.*;
import java.text.SimpleDateFormat;

class User {
    private String userId;
    private String name;
    private int age;
    private String email;
    private String contactNumber;

    public User(String userId, String name, int age, String email, String contactNumber) {
        validateEmail(email);
        validateContactNumber(contactNumber);

        this.userId = userId;
        this.name = name;
        this.age = age;
        this.email = email;
        this.contactNumber = contactNumber;
    }

    private void validateEmail(String email) {
        if (!email.matches("^[A-Za-z0-9._%+-]+@gmail\\.com$")) {
            throw new IllegalArgumentException("Invalid Gmail address.");
        }
    }

    private void validateContactNumber(String contactNumber) {
        if (!contactNumber.matches("\\d{10}")) {
            throw new IllegalArgumentException("Invalid contact number. Must be 10 digits.");
        }
    }

    public String getUserId() { return userId; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getEmail() { return email; }
    public String getContactNumber() { return contactNumber; }

    @Override
    public String toString() {
        return userId + "," + name + "," + age + "," + email + "," + contactNumber;
    }

    public static User fromString(String data) {
        String[] fields = data.split(",");
        return new User(fields[0], fields[1], Integer.parseInt(fields[2]), fields[3], fields[4]);
    }
}

abstract class HealthRecord {
    protected String recordId;
    protected Date date;
    protected String notes;
    private static final SimpleDateFormat DATE_FORMAT = new SimpleDateFormat("yyyy-MM-dd");

    public HealthRecord(String recordId, Date date, String notes) {
        this.recordId = recordId;
        this.date = date;
        this.notes = notes;
    }
```

```java

    public String getRecordId() { return recordId; }
    public Date getDate() { return date; }
    public String getNotes() { return notes; }

    public static String formatDate(Date date) {
        return DATE_FORMAT.format(date);
    }

    public static Date parseDate(String date) throws Exception {
        return DATE_FORMAT.parse(date);
    }

    public abstract String getRecordDetails();
    public abstract String toDataString();

    public static HealthRecord fromString(String data) throws Exception {
        String[] fields = data.split(",");
        Date date = parseDate(fields[1]);
        if (fields[0].equals("Medication")) {
            return new Medication(fields[0], date, fields[4], fields[2], fields[3], fields[5]);
        } else {
            return new DoctorConsultation(fields[0], date, fields[4], fields[2], fields[3]);
        }
    }
}

class Medication extends HealthRecord {
    private String medicationName;
    private String dosage;
    private String frequency;

    public Medication(String recordId, Date date, String notes, String medicationName, String dosage, String frequency) {
        super(recordId, date, notes);
        this.medicationName = medicationName;
        this.dosage = dosage;
        this.frequency = frequency;
    }

    @Override
    public String getRecordDetails() {
        return "Medication: " + medicationName + "\nDosage: " + dosage + "\nFrequency: " + frequency + "\nDate: " + date + "\nNotes: " + notes;
    }

    @Override
    public String toDataString() {
        return "Date:" + formatDate(date) + "\nMedication: " + medicationName + "\nDosage: " + dosage + "\nFrequency: " + frequency + "\nDate: " + date + "\nNotes: " + notes;
    }
}

class DoctorConsultation extends HealthRecord {
    private String doctorName;
    private String specialization;

    public DoctorConsultation(String recordId, Date date, String notes, String doctorName, String specialization) {
        super(recordId, date, notes);
        this.doctorName = doctorName;
        this.specialization = specialization;
    }

    @Override
    public String getRecordDetails() {
        return "Doctor: " + doctorName + "\nSpecialization: " + specialization +  "\nDate: " + date + "\nNotes: " + notes;
    }

    @Override
    public String toDataString() {
        return "DoctorConsultation," + formatDate(date) + "," + doctorName + "," + specialization + "," + notes;
    }
}

class RecordService {
    private static final String RECORD_DIRECTORY = "records/";

    public RecordService() {
        createRecordDirectory();
    }

    public void addRecord(String userId, HealthRecord record) {
        saveRecordToFile(userId, record);
        System.out.println("Record added successfully for user " + userId + "!");
    }

    public void viewRecord(String userId) {
        File recordFile = new File(RECORD_DIRECTORY + userId + ".txt");
        if (recordFile.exists()) {
            try (Scanner scanner = new Scanner(recordFile)) {
                while (scanner.hasNextLine()) {
                    System.out.println(scanner.nextLine());
                }
            } catch (FileNotFoundException e) {
                System.out.println("Record file not found for user " + userId + ".");
            }
        } else {
            System.out.println("No record found for user ID: " + userId);
        }
    }
```

```java
    private void createRecordDirectory() {
        File directory = new File(RECORD_DIRECTORY);
        if (!directory.exists()) {
            directory.mkdirs();
        }
    }

    private void saveRecordToFile(String userId, HealthRecord record) {
        try (PrintWriter writer = new PrintWriter(new FileWriter(RECORD_DIRECTORY + userId + ".txt", true))) {
            writer.println(record.toDataString());
        } catch (IOException e) {
            System.out.println("Error saving record for user " + userId + ": " + e.getMessage());
        }
    }
}

public class PersonalHealthRecordSystem {
    private static UserService userService = new UserService();
    private static RecordService recordService = new RecordService();
    private static Scanner input = new Scanner(System.in);

    public static void main(String[] args) {
        boolean shouldRun = true;

        while (shouldRun) {
            System.out.println("Select an option: 1) Add User 2) Add Record 3) View Records 4) Exit");
            int option = input.nextInt();
            input.nextLine();

            switch (option) {
                case 1:
                    addUser();
                    break;
                case 2:
                    addRecord();
                    break;
                case 3:
                    System.out.println("Enter User ID to view records:");
                    String userId = input.nextLine();
                    recordService.viewRecord(userId);
                    break;
                case 4:
                    shouldRun = false;
                    System.out.println("Exiting...");
                    break;
                default:
                    System.out.println("Invalid option.");
            }
        }
    }

    private static void addUser() {
        System.out.println("Enter User ID:");
        String userId = input.nextLine();
        System.out.println("Enter name:");
        String name = input.nextLine();
        System.out.println("Enter age:");
        int age = input.nextInt();
        input.nextLine();
        System.out.println("Enter email:");
        String email = input.nextLine();
        System.out.println("Enter contact number:");
        String contactNumber = input.nextLine();

        try {
            User user = new User(userId, name, age, email, contactNumber);
            userService.addUser(user);
        } catch (IllegalArgumentException e) {
            System.out.println("Error adding user: " + e.getMessage());
        }
    }

    private static void addRecord() {
        System.out.println("Enter User ID:");
        String userId = input.nextLine();
        System.out.println("Enter Record ID:");
        String recordId = input.nextLine();
        System.out.println("Enter date (yyyy-MM-dd):");
        String dateString = input.nextLine();
        System.out.println("Enter notes:");
        String notes = input.nextLine();
        System.out.println("Enter record type (1 for Medication, 2 for Doctor Consultation):");
        int type = input.nextInt();
        input.nextLine();
```

```java
        try {
            Date date = HealthRecord.parseDate(dateString);
            HealthRecord record;

            if (type == 1) {
                System.out.println("Enter medication name:");
                String medicationName = input.nextLine();
                System.out.println("Enter dosage:");
                String dosage = input.nextLine();
                System.out.println("Enter frequency:");
                String frequency = input.nextLine();
                record = new Medication(recordId, date, notes, medicationName, dosage, frequency);
            } else if (type == 2) {
                System.out.println("Enter doctor name:");
                String doctorName = input.nextLine();
                System.out.println("Enter specialization:");
                String specialization = input.nextLine();
                record = new DoctorConsultation(recordId, date, notes, doctorName, specialization);
            } else {
                System.out.println("Invalid record type.");
                return;
            }

            recordService.addRecord(userId, record);
        } catch (Exception e) {
            System.out.println("Error adding record: " + e.getMessage());
        }
    }
}

class UserService {
    private Map<String, User> users = new HashMap<>();

    public void addUser(User user) {
        if (users.containsKey(user.getUserId())) {
            System.out.println("User ID already exists.");
        } else {
            users.put(user.getUserId(), user);
            System.out.println("User added successfully.");
        }
    }
}
```

OUTPUT

>Add User

```
Select an option: 1) Add User 2) Add Record 3) View Records 4) Exit
1
Enter User ID:
61
Enter name:
vedant
Enter age:
20
Enter email:
thumbadiya.vedant@gmail.com
Enter contact number:
8141624406
User added successfully.
```

>Add Record

```
Select an option: 1) Add User 2) Add Record 3) View Records 4) Exit
2
Enter User ID:
61
Enter Record ID:
324
Enter date (yyyy-MM-dd):
2024-11-04
Enter notes:
take rest
Enter record type (1 for Medication, 2 for Doctor Consultation):
1
Enter medication name:
fever
Enter dosage:
dolo
Enter frequency:
2
Record added successfully for user 61!
```

>61.text (file handling)

61.txt

File Edit View

```
Date:2024-11-04
Medication: fever
Dosage: dolo
Frequency: 2
Date: Mon Nov 04 00:00:00 IST 2024
Notes: take rest
```

Ln 1, Col 1 | 113 characters | 180% | Unix (LF) | UTF-8

>View Records

```
Select an option: 1) Add User 2) Add Record 3) View Records 4) Exit
3
Enter User ID to view records:
61
Date:2024-11-04
Medication: fever
Dosage: dolo
Frequency: 2
Date: Mon Nov 04 00:00:00 IST 2024
Notes: take rest
```

>Exception Handling

```
Select an option: 1) Add User 2) Add Record 3) View Records 4) Exit
1
Enter User ID:
324
Enter name:
shyam
Enter age:
20
Enter email:
shyam
Enter contact number:
9426554406
Error adding user: Invalid Gmail address.
```

CONCLUSION

The Health Record System is a comprehensive Java application designed to manage patient health records efficiently. It allows users to add, search and view health records, providing a user-friendly interface for managing critical patient information. The application emphasizes data integrity by validating email addresses and ensuring positive integer inputs for age. It incorporates file handling to persist records, allowing them to be saved individually in text files. Overall, this project illustrates effective software design principles, including modularity, user input validation, and data management, creating a solid foundation for a health record management system.